

Lab Report: Parallel Monte Carlo Pi Benchmarks

Nurdaulet Serketay | ID: 230103358 01-N 03-P

Part 1: The Phantom Bug

Results of 5 consecutive runs (50,000,000 points, 4 threads, raw totalHits++):

- Run 1: $\pi = 0.785301$ (Hits: 9,816,265)
- Run 2: $\pi = 0.785492$ (Hits: 9,818,649)
- Run 3: $\pi = 0.785412$ (Hits: 9,817,653)
- Run 4: $\pi = 0.785413$ (Hits: 9,817,661)
- Run 5: $\pi = 0.785482$ (Hits: 9,818,528)

The code gave approximately 0.785 (instead of approximately 3.1415) because of a data race. Multiple threads simultaneously modified totalHits in un-synchronized memory, overwriting each other's increments.

Part 2: The Synchronization Trap

Performance comparison on 50,000,000 points:

- Single-threaded: $\pi = 3.14203$, Runtime = 477 ms
- Atomic (4 threads): $\pi = 3.14162$, Runtime = 1043 ms

While `std::atomic` fixed the data race and produced an accurate value of Pi, it ran more than 2x slower than the single-threaded version due to memory bus locking and cache line contention.

Part 3: OpenMP-Style Reduction

Benchmark results for 100,000,000 iterations:

- 1 Thread: Runtime = 917.0 ms, Speedup = 1.00x, Efficiency = 100.0%
- 2 Threads: Runtime = 502.0 ms, Speedup = 1.83x, Efficiency = 91.3%
- 4 Threads: Runtime = 263.0 ms, Speedup = 3.49x, Efficiency = 87.2%
- 8 Threads: Runtime = 187.0 ms, Speedup = 4.90x, Efficiency = 61.3%
- 16 Threads: Runtime = 182.0 ms, Speedup = 5.04x, Efficiency = 31.5%
- 32 Threads: Runtime = 184.0 ms, Speedup = 4.98x, Efficiency = 15.6%

Questions & Answers

Question 1: Look at your row for 16 threads. Why didn't your CPU run twice as fast as 8 threads?

My MacBook Air CPU has a physical limit of 8 hardware cores (4 Performance cores + 4 Efficiency cores). Creating 16 or 32 threads does not double the physical execution units available. Instead, excess threads compete for the same 8 cores, introducing overhead for thread context switching and scheduling without providing additional compute power. Additionally, Efficiency cores run at lower clock frequencies, causing sub-linear scaling beyond 4 Performance cores.

Question 2: Why was the synchronized version in Part 2 slower than running on one single core?

Every atomic write (`atomicHits.fetch_add`) causes Cache Line Bouncing between core L1/L2 caches via the cache coherence protocol. The CPU cores spend ~95% of their execution cycles stalled on memory bus locks waiting for access permission rather than doing actual arithmetic.